RAW_Image 等待期行动指南

> 生成日期：2026-09-21 背景：学长反馈当前采集设备不支持 RAW 导出，新双目设备（FLIR BFS-U3-16S2C-CS）搭建中，真实标定参数与 RAW 数据暂不可得。 目标：设备到货前把所有能提前做的准备全部做完，设备一到即可在 1~2 天内跑通真实链路。

〇、硬件变更对项目的影响（先说结论）

新相机关键规格（FLIR BFS-U3-16S2C-CS）

项目	规格	对项目的影响
传感器	Sony IMX273，1/2.9"，3.45μm，**全局快门**	全局快门无果冻效应，对双目立体匹配是**利好**
分辨率	1440×1080（4:3），最高 226 FPS（8bit）	下游要求 1280×720（16:9），管线输出时会**拉伸变形**，需要决定：裁剪 16:9 区域 or 改下游分辨率约定（建议与学长确认，或标定时直接标 ROI 模式）
ADC	10 / 12 bit	白电平 1023 / 4095，管线按 12bit 配置
动态范围	71.43 dB	下游要求 70~80 dB，**单帧刚好及格**；HDR 场景（隧道口）将来需要多曝光融合
接口	USB 3.1 Gen1（USB3 Vision 协议）	采集走 Spinnaker SDK，不是 rawpy
GPIO	4 路（1 路光耦入、1 路光耦出、2 路非隔离）	双目同步靠 GPIO 主从触发（见任务三）
片上处理	色彩校正/Gamma/饱和度/锐化（可用节点关闭）	全部关闭，只取裸 Bayer
镜头	CS 口，需另配镜头	1/2.9" 靶面，配镜头时焦距按视场角算（见任务三）

三个必须提前想清楚的问题

1. **工业相机没有 rawpy 元数据**：黑电平、白平衡、色彩矩阵都要自己解决（黑电平从相机节点读，白平衡用管线里现成的灰世界算法，色彩矩阵先用单位阵）——所以"代码预留"任务（任务四）非常必要。
2. **USB3 带宽**：1440×1080 的 12bit 帧约 3.1 MB。两路 60 FPS ≈ 187 MB/s×2，接近单控制器极限。**建议两相机分别插不同 USB3 控制器**（或降帧率），采集中断是工业相机最常见的坑。
3. **双目同步**：USB3 相机没有 PTP 精密时间协议（那是 GigE 的功能），必须用 **GPIO 主从触发或外部触发板**（E-SIP 大纲里正好规划了同步触发板，可以和学长确认）。软件触发抖动毫秒级，不满足 <1ms 要求。

任务一（P0，本周就做）：Spinnaker SDK 下载安装 + 文档阅读

1.1 下载安装（现在就能做，不需要相机）

1. 到 FLIR 官网下载 **Spinnaker SDK**（需注册免费账号）： https://www.flir.com/products/spinnaker-sdk/
   - 选 Windows 完整版（含 SpinView 调试工具 + Python 支持）
2. 安装后给 `adaptiveisp` 环境装 Python 绑定（版本必须与 SDK 一致）：

```
conda activate adaptiveisp
pip install spinnaker-python
# 或直接安装 SDK 目录自带的 wheel:
# pip install "C:\Program Files\Spinnaker\spinnaker_python-*.whl"
```

3. 验证：`python -c "import PySpin; print(PySpin.SYSTEM_VERSION)"`

1.2 文档阅读清单（按顺序）

#	文档	重点读什么	链接
1	相机规格与帧率表	分辨率/帧率/ADC/GPIO 定义	https://softwareservices.flir.com/BFS-U3-16S2/latest/Model/spec.html
2	PySpin Python 编程指南	采集流程、GetNDArray、节点读写	https://softwareservices.flir.com/spinnaker/latest/guides/python-programmer-guide.html
3	官方示例 **StereoGPIO**	双相机 GPIO 同步的官方配置方法	Spinnaker 安装目录 `Examples/Python/` 下，或官网示例库
4	官方示例 Trigger / Acquisition	硬件触发、连续采集写法	同上
5	StereoSync 开源项目	两台 BFS 相机外部触发同步的完整参考	https://github.com/murraiya/StereoSync
6	IMX273 传感器手册（可选，跟学长要）	黑电平典型值、光谱响应、暗电流	索尼官网/FLIR 文档

阅读产出物（写进 WORKLOG.md）：一份《Spinnaker 采集要点笔记》，包括：Bayer 像素格式命名（BayerRG8/12/16）、GetNDArray 返回的位深行为、GPIO 线路图、时间戳 API 名称。

任务二（P1，最高价值，本周完成）：检测模型接入（纯软件演示）

> 这是**唯一不依赖任何真实硬件**就能产出"视频目标检测演示"成果的任务，直接对应学长的终极要求。

2.1 环境事实（已核实）

你的 `yolo` conda 环境已装好 **ultralytics 8.4.47 + torch 2.11.0（CPU 版）**，马上可用。CPU 跑 YOLOv8n 在 1280×720 上约 5~15 FPS——**离线处理视频足够**；以后上了 MyGPU（有 GPU）再换 CUDA 版 torch，实时性自然解决。

2.2 改造方案：给 video_process.py 加 --detect 参数

在现有脚本上增加：

```
# 参数
parser.add_argument('--detect', type=str, default=None,
                    help='检测模型权重路径，如 yolov8n.pt（None=不检测）')
parser.add_argument('--detect-side', choices=['left', 'right', 'both'],
default='both')

# 初始化（放循环外，模型只加载一次）
detector = YOLO(args.detect) if args.detect else None

# 每帧处理（在 rectifier 之后、写视频之前）
if detector is not None:
    if args.detect_side in ('left', 'both'):
        left_srgb = draw_detections(left_srgb, detector(left_srgb, verbose=False))
    ...
```

`draw_detections()` 就是 `results[0].plot()`（ultralytics 自带画框函数）。

2.3 立即可跑的演示命令（实测可行的链路）

```
# 1) 激活 yolo 环境
conda activate yolo

# 2) 下载预训练权重（第一次会自动下载，或手动：
#    https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n.pt）
# 3) 用任意 RGB 视频（手机拍的街景/行人视频都行）跑：
python scripts/video_process.py \
    --left 任意视频.mp4 --right 任意视频.mp4 \
    --output data/output_video --no-rectify \
    --detect yolov8n.pt
```

产出：**带检测框的左右拼接演示视频**。这就是给学长看的第一个"视频效果展示目标检测"成果，比等硬件快得多。

> 注意：video_process.py 跑在 adaptiveisp 环境（rawpy/管线依赖），检测需要 ultralytics——两个环境二选一。最省事方案：**把 ultralytics 也装进 adaptiveisp**（`pip install ultralytics`，CPU 版 torch 会自动装），一个环境搞定全链路；`yolo` 环境留作备用。

任务三（P1）：双目硬件同步方案预研

3.1 推荐方案：GPIO 主从触发（成本 0，一根杜邦线）

```
相机A(主) Line1 配置为 Output, Source = ExposureActive
        │ (信号线)
相机B(从) Line0 配置为 Input, TriggerMode = On, TriggerSource = Line0
```

- 相机 A 曝光期间输出高电平 → 相机 B 同步开始曝光，两帧时间差 = 曝光时间级（μs 级），远优于 <1ms 要求；
- 关键节点（写入你的笔记，设备到货直接配置）：
  - 主：`LineSelector=Line1, LineMode=Output, LineSource=ExposureActive`
  - 从：`LineSelector=Line0, LineMode=Input, TriggerMode=On, TriggerSource=Line0, TriggerSelector=FrameStart`
- 备选：外部触发板（E-SIP 大纲里的同步触发板）同时触发两机，更稳，但依赖实验室硬件。

3.2 其他硬件准备

- **镜头**：CS 口 + 1/2.9" 靶面（传感器 4.97×3.73mm）。视场角公式 `FOV_H = 2·atan(2.48/f)`：要 60° 水平视场约配 f=4.3mm 镜头。提醒学长采购时注意靶面尺寸要 ≥1/2.9"，否则暗角。
- **USB3 线缆与接口**：优质带屏蔽 USB3 线（≤3m），两相机插不同 USB3 控制器。
- **刚性支架**：双目基线固定（建议 10~15cm 量级，具体按检测距离），机械结构稳固（标定后外参会因振动漂移）。

任务四（P1）：代码预留——为 FLIR 裸 Bayer 流做抽象

4.1 现状分析（为什么不用大改）

好消息：`ISPPipeline.process()` 接受的是 `raw_data + metadata` 字典，`process_bayer_data()` 已经支持"外部 Bayer 数据直接进管线"。**管线本身不用动**，需要做的是"数据入口"抽象：

```
现有:  rawpy 读 .ARW 文件 ──> RawReader ──> ndarray + metadata ──> 管线
将来:  Spinnaker 相机流 ────> ???       ──> ndarray + metadata ──> 管线
```

4.2 建议的抽象设计（新文件 `src/isp_pipeline/raw_source.py`）

```python
@dataclass
class RawFrame:
    """统一的原始帧数据结构：无论来自文件还是相机"""
    bayer: np.ndarray              # HxW Bayer 阵列
    bayer_pattern: str             # 'RGGB' 等
    bit_depth: int                 # 10 / 12 / 8
    black_level: int = 0
    white_level: int = 4095
    exposure_us: float = 0.0
    gain_db: float = 0.0
    timestamp_ns: int = 0          # 设备时间戳，双目同步验证用
```

```
    frame_id: int = 0

class RawSource(abc.ABC):
    """帧源抽象：文件 / 相机统一接口"""
    @abc.abstractmethod
    def __next__(self) -> RawFrame: ...

class RawpyFileSource(RawSource):   # 包装现有 raw_reader 逻辑
class SpinnakerStereoSource(RawSource):  # 双相机同步采集（骨架，设备到货后填充）
```

`SpinnakerStereoSource` 骨架现在就写（用 mock 数据测试通过），设备到货后只填采集细节：Init → 配 PixelFormat=BayerRG12 → 关片上处理（ColorTransformation/Gamma/Sharpness 全部 Disable）→ BeginAcquisition → 主从触发 → 双路 GetNextImage → 组帧。

4.3 FLIR Bayer 命名映射（写进代码注释里）

FLIR 像素格式	本项目 Bayer 模式	位深
BayerRG8 / BayerRG12 / BayerRG16	RGGB	8/12/16
BayerGB8/12/16	GBRG	同
BayerGR8/12/16	GRBG	同
BayerBG8/12/16	BGGR	同

4.4 三个易错点（提前记下，设备到货一次到位）

1. **12-bit 的容器**：PySpin 的 BayerRG12 通常放在 16-bit 数组里，数值可能在低 12 位（0~4095）**或已左移**——拿到相机后第一件事写个 3 行脚本打印 `img.max()` 确认，再定 white_level。
2. **黑电平**：IMX273 的黑电平从相机节点 `BlackLevel` 读（不要硬编码，不同批次有差异）；同时对比 datasheet 典型值。
3. **时间戳**：`image.GetTimeStamp()` 是设备时间戳（ns）。双路采集后统计左右时间戳差，直接验证硬件同步是否达标（<1ms）。

任务五（P2）：标定预研（设备到货即可上手）

5.1 现在就能做的：双目标定"开卷考试"脚本

现有 `validate_calibration.py` 只验证了单目标定数学。照葫芦画瓢写 **`scripts/stereo_calibration_synth_test.py`**：

- 设定真值 K_left/K_right/R/T（例如基线 0.12m）；
- 用 `cv2.projectPoints` 生成 20 对"左右同步观测的棋盘格角点"（加一点检测噪声更真实）；
- 喂给 `calibrator.py` 里的 `StereoCalibrator`；
- 对比输出的 R/T/内参与真值的误差，验证 `StereoCalibrator` 全流程正确。 产出物：验证通过 + 一个可复用的"合成数据→标定→YAML"完整脚本模板。

5.2 提前准备实物棋盘格

- **规格建议**：9×6 内角点、方格 25mm（A4 打印正好）、**亚光**纸打印、平整粘贴到硬板（玻璃/铝板）上；误差小于 0.1mm 最好。
- 进阶：打印 **ChArUco 板**（OpenCV 支持），双目拍摄有遮挡/出界时检测更稳，强烈推荐。
- 打印好后放在实验室，设备到货直接开标。

5.3 双目标定 SOP 要点（提前消化，写进 WORKLOG）

- 至少 15~20 对**同步**拍摄的棋盘格图，姿态要求：覆盖画面四角 + 不同距离 + 不同倾角（含一点旋转，避免全是正面）；
- 左右都必须完整检测到棋盘格才保留该对；
- 标定目标：单目重投影误差 < 0.5px，双目极线误差 < 0.5px；
- 标定结果字段 → 项目 YAML 的映射（`StereoCalibrator.save_yaml` 输出格式已对齐，注意 T 单位是米）。

任务六（P2，可选）：AODRaw 真实 RAW 验证管线

- 下载 AODRaw 的**切片版（1280×1280）或降采样版**少量样本（几个 GB 级别，不用全下），用 `pipeline.process_file()` 跑通 rawpy 路径，肉眼检查色彩/曝光合理性；
- 意义："真实传感器数据 + 完整元数据"下管线的色彩链路（黑电平/白平衡/CCM/Gamma）是否合理——**这是设备到货前唯一能碰到的真实 RAW**；
- 注意：AODRaw 是索尼 A7M4 单目数据，与 FLIR 裸 Bayer 不同，只能验证 ISP 链路，不能验证采集与同步；
- 数据集链接：https://github.com/lzyhha/AODRaw（百度网盘/Google Drive 下载）。

任务七（P2）：MyGPU 服务器部署预演（不依赖硬件）

1. `git push origin feature/fix-core-bugs`（走代理）；
2. 服务器上 clone → `conda create -n binocular_isp python=3.10` → `pip install -r requirements.txt` → 跑 `tests/test_pipeline.py`；
3. 把任务二的**检测演示视频**搬到服务器跑一遍（Linux 上装 ultralytics + CUDA torch），记录 GPU 推理速度；
4. 产出：一份《服务器部署记录》（环境、命令、坑）写进 WORKLOG.md。

> 依赖学长给服务器账号，可以先问。

时间线总览（"不要闲下来"版）

周次	主线任务	产出物
本周	① Spinnaker SDK 安装 + 文档阅读（任务一）② 检测模型接入 + 纯软件演示视频（任务二）	带检测框的演示视频 + 《Spinnaker 要点笔记》
下周	③ RawFrame 抽象 + Spinnaker 骨架代码（任务四）④ 双目标定开卷考试脚本（任务五）⑤ 打印棋盘格	可运行骨架代码 + 标定验证脚本 + 实物棋盘格
第三周	⑥ AODRaw 真实 RAW 验证（可选）⑦ 服务器部署预演（任务七）⑧ 双目同步方案定稿（任务三）	服务器部署记录 + 同步接线方案图

周次	主线任务	产出物
设备到货	装驱动 → SpinView 单机出图 → PySpin 采集 → GPIO 同步验证 → 实拍标定 → 真实链路跑通 → 视频演示	学长的终极要求达成

本指南基于 FLIR BFS-U3-16S2C-CS 官方规格与 Spinnaker SDK 官方文档编写；所有"环境已核实"项均在本机实测。